

Compte rendu de la semaine 5

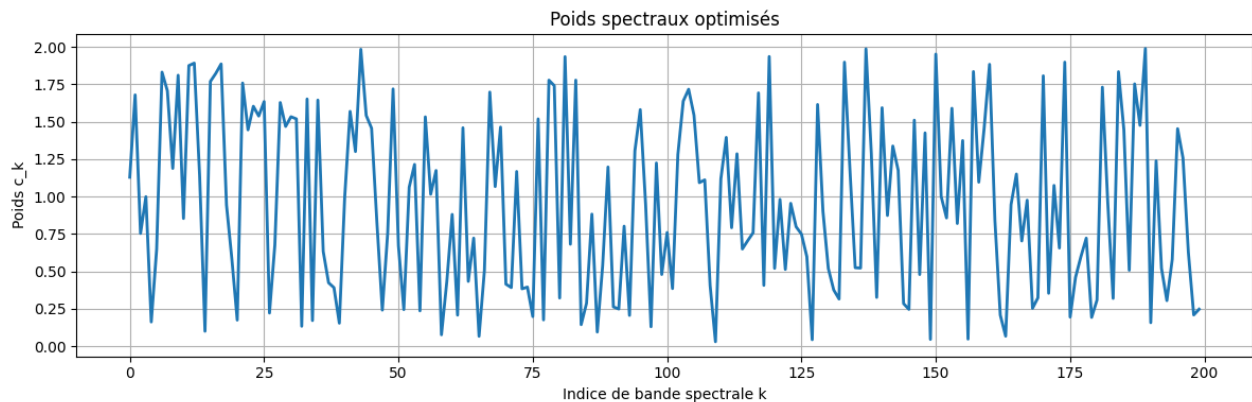
Sujet du stage : Classification non supervisée d'une image satellite hyperspectrale

Travail effectué durant la semaine

Cette semaine, j'ai travaillé principalement sur la fonction d'optimisation. Je suis parti de la littérature pour comprendre la méthode du recuit simulé ainsi que son implémentation, puis j'ai essayé de réaliser une expérience d'optimisation des poids spectraux c_k . J'ai également effectué une simulation avec 20 000 itérations, qui m'a pris plusieurs heures. Pour pallier ce problème de temps de calcul, je travaille sur deux machines en parallèle, ce qui me permet de lancer une simulation et de la laisser tourner sans bloquer mon travail. Je vais maintenant détailler l'ensemble des expériences abordées cette semaine.

Expérience 1

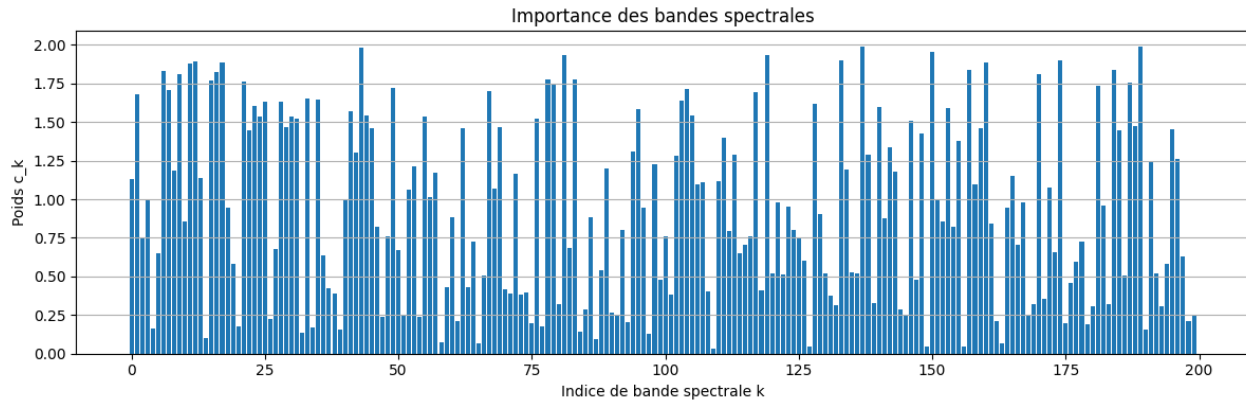
Courbe poids spectraux c_k



Les poids oscillent de façon très irrégulière entre 0 et 2 sur toutes les 200 bandes, sans aucune structure visible. On peut noter bien que y a certaines bandes clairement favorisées (poids proches de 2) et d'autres écartées (poids proches de 0).

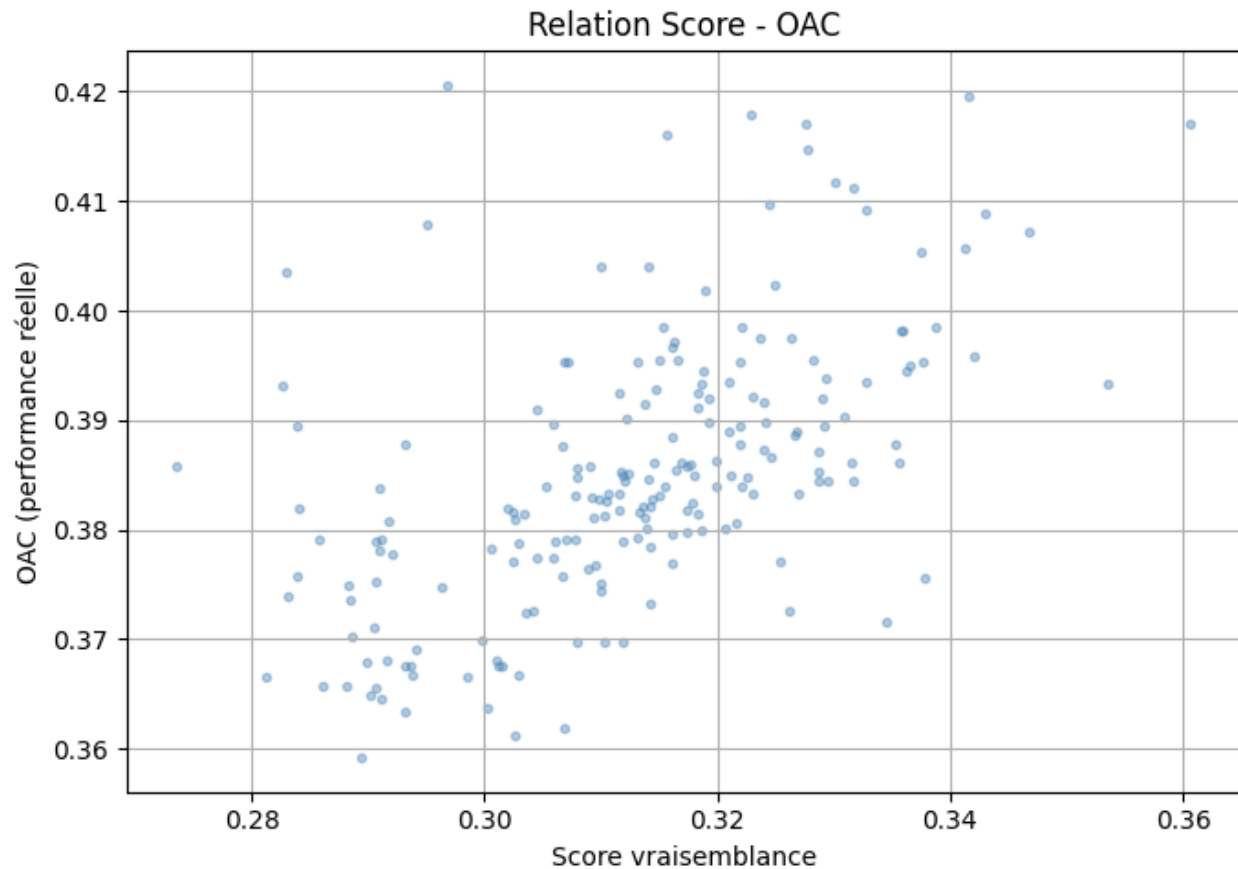
La courbe met en évidence une **absence de structure lisse ou régulière** : les poids ne suivent pas une tendance progressive entre bandes adjacentes.

Importance des bandes (barres)



C'est la même information que le graphique 1 mais sous forme de barres, ce qui confirme la même conclusion : Y a des bandes spectrales qui se distingue clairement. La distribution est quasi-uniforme sur $[0, 2]$. Ce graphique aurait été intéressant selon moi si on avait observé, par exemple, que les bandes dans un intervalle donné ont systématiquement des poids élevés, cela aurait indiqué que ces longueurs d'onde sont plus importantes pour cette image. Pour l'instant, on ne peut rien conclure.

Relation Score de vraisemblance $\leftrightarrow$ OAC

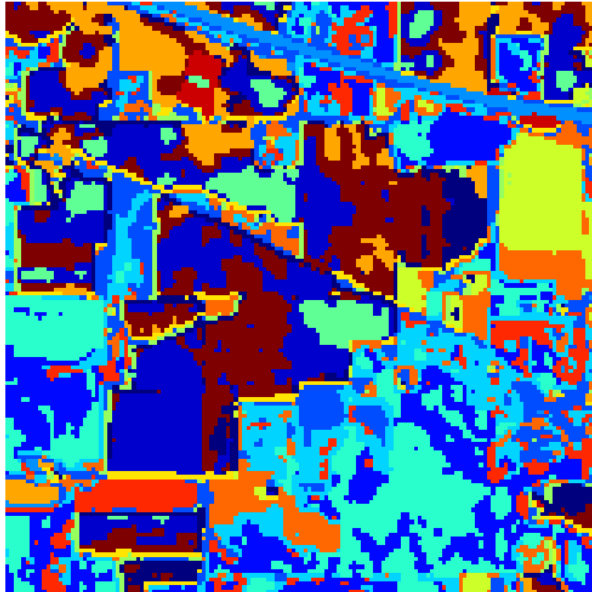


On observe une **tendance positive très légère** : quand le score de vraisemblance augmente (vers la droite), l'OAC a tendance à être légèrement plus élevé. Mais la dispersion est très grande pour un même score autour de 0.32, l'OAC varie de 0.37 à 0.42. Cela signifie deux choses. D'abord, la vraisemblance guide bien dans la bonne direction, ce qui est encourageant et valide l'approche. Ensuite, cette guidance est faible, cette vraisemblance n'explique qu'une petite partie de la variation de l'OAC, ce qui suggère qu'il faudrait une mesure de vraisemblance plus discriminante, ou davantage d'itérations pour que le signal émerge du bruit.

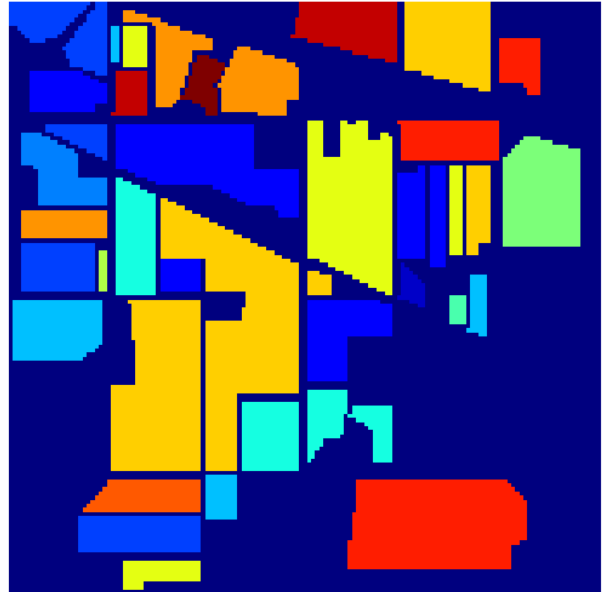
Expérience 2

Cette 2eme expérience, c'est principalement les 20.000 itérations. On a pas toujours le resultat voulu, les régions ne sont pas toujours homogènes. L'OAC est passé de 0.3955 à 0.4078 et le CE de 0.679 à 0.629, ce qui confirme que la vraisemblance guide bien la recherche dans la bonne direction, mais de façon très inefficace.

Meilleure classification
OAC=0.4078 | CE=0.6287



Ground Truth

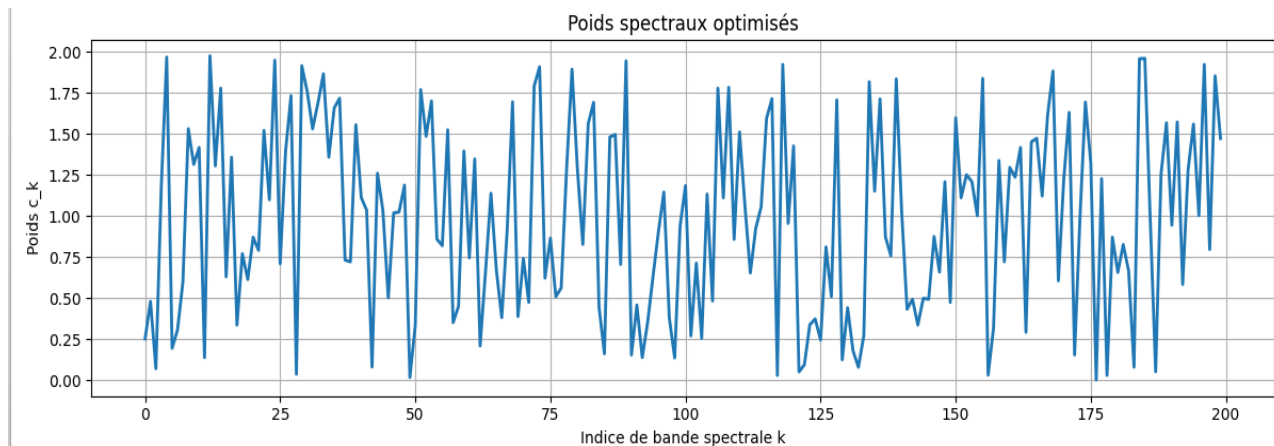


```
PS C:\Users\Assane\Documents\Internship\code> python .\kmeans_vers2.py
=== Optimisation en cours ===
Iter 000 | CE=0.6790 | OAC=0.3955 | score=0.3167
Iter 001 | CE=0.6652 | OAC=0.4116 | score=0.3301 amélioration
Iter 004 | CE=0.6426 | OAC=0.3934 | score=0.3535 amélioration
Iter 061 | CE=0.6349 | OAC=0.4170 | score=0.3607 amélioration
Iter 2915 | CE=0.6287 | OAC=0.4078 | score=0.3670 amélioration

=== Résultat final ===
OAC = 0.4078 | CE = 0.6287
```

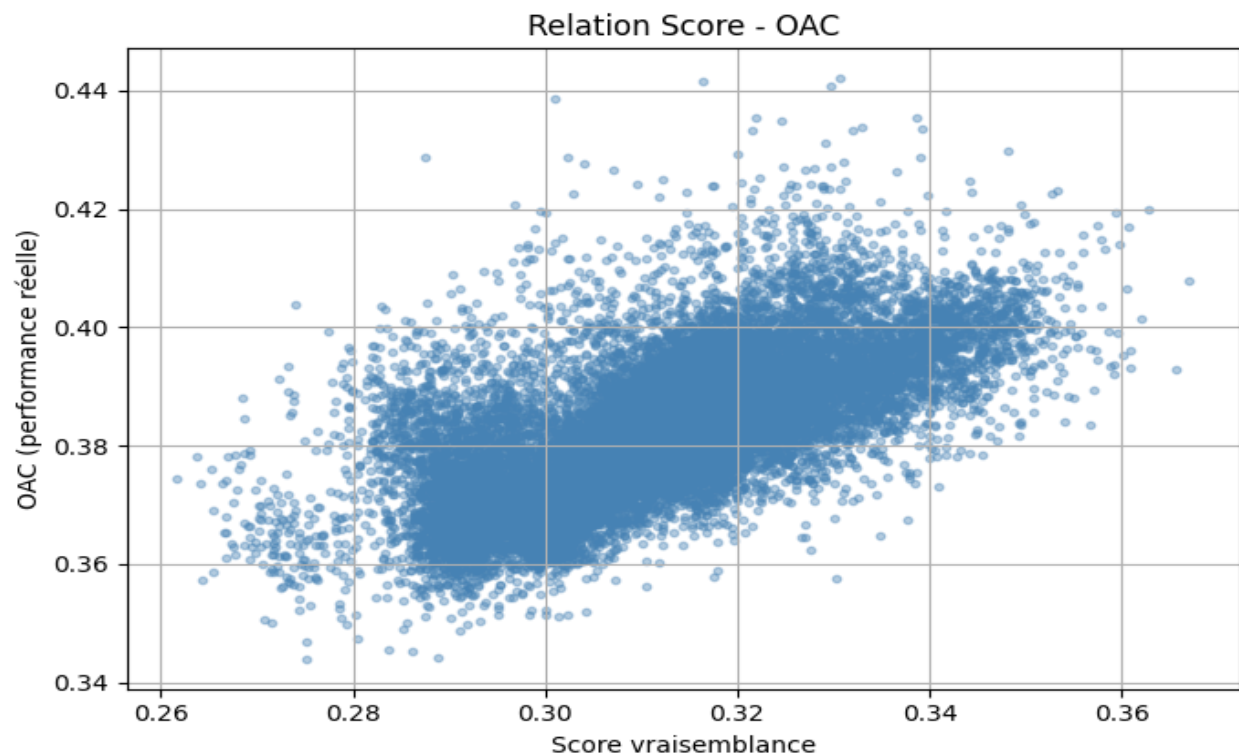
Comparaison des poids spectraux (20 000 vs 200 itérations)

Dans les deux cas, les poids oscillent de façon irrégulière entre 0 et 2 sur toutes les bandes. On aurait espéré voir certaines bandes systématiquement favorisées, mais ce n'est pas le cas. Et surtout, les deux courbes se ressemblent beaucoup malgré un facteur 100 dans le nombre d'itérations. C'est la limite fondamentale de l'approche actuelle



Relation Score de vraisemblance $\leftrightarrow$ OAC (20 000 itérations)

Le nuage est dispersé : pour un score de vraisemblance de 0.32 par exemple, l'OAC varie de 0.36 à 0.42. Cela suggère que le score combiné $(1-CE) \times (1-CIP)$ est légèrement moins informatif. Une piste serait d'examiner si un de ces indicateurs apporte vraiment une information complémentaire à l'autre, ou s'il ajoute simplement du bruit à la mesure.



Expérience 3

Implémentation du recuit simulé

Principe du recuit simulé (d'après ma compréhension)

L'hypothèse est :

Si un vecteur de poids donne une bonne vraisemblance, alors des vecteurs proches ont probablement eux aussi une bonne vraisemblance.

On effectue donc une exploration locale : $C_{new} = C_{best} + \varepsilon$ (avec $\varepsilon \sim N(0, \sigma^2)$)

L'algorithme implémente une recherche locale stochastique dans l'espace des poids spectraux. À chaque itération, un nouveau vecteur est obtenu par perturbation gaussienne du meilleur vecteur courant. L'amplitude de cette perturbation est contrôlée par un paramètre σ décroissant, permettant une transition progressive d'une phase d'exploration globale vers une phase d'exploitation locale.

J'ai essayé l'implémentation ci-dessous et vous pouvez retrouver le code via mon dépôt github : https://github.com/assanethiao/Internship/blob/master/code/kmeans_vers3.py

```

# RECRUIT SIMULÉ pour optimiser les poids c_k

def optimize_weights_recuit(data_norm, gt, n_clusters=16, n_iter=500, seed=0):
    """
    Recruit simulé pour optimiser les poids spectraux c_k.

    Idée : au lieu de tirer des poids complètement aléatoires,
    on explore AUTOUR des meilleurs poids trouvés, avec un pas
    qui diminue progressivement (sigma décroissant).
    """
    rng = np.random.RandomState(seed)
    K = data_norm.shape[2]

    # Initialisation : poids uniformes
    best_weights = np.ones(K)
    X0 = build_features(data_norm, best_weights)
    best_classif = run_kmeans(X0, n_clusters)
    best_score = compute_vraisemblance(best_classif)
    best_ce = compute_CE(best_classif)
    best_oac = clustering_accuracy(gt, best_classif)

    # Historiques
    hist_ce = [best_ce]
    hist_oac = [best_oac]
    hist_score = [best_score]
    hist_sigma = []

    # Paramètres du recuit
    sigma_init = 1.0 # écart-type initial pour l'exploration
    sigma_min = 0.001 # écart-type minimal
    n_plateau = 0 # compteur d'itérations sans amélioration
    sigma = sigma_init

    print(f"Iter 000 | CE={best_ce:.4f} | OAC={best_oac:.4f} " f"| score={best_score:.4f} | sigma={sigma:.4f}")

    for t in range(1, n_iter + 1):

        # Exploration autour des meilleurs poids
        # avec un bruit gaussien de sigma décroissant
        noise = rng.randn(K) * sigma
        new_weights = best_weights + noise
        new_weights = np.maximum(new_weights, 0) # pas de poids négatifs

        # Fallback si tous les poids sont nuls
        if new_weights.sum() == 0:
            new_weights = np.ones(K)

        X_new = build_features(data_norm, new_weights)
        classif = run_kmeans(X_new, n_clusters, seed=t)
        score = compute_vraisemblance(classif)
        ce = compute_CE(classif)
        oac = clustering_accuracy(gt, classif)

        hist_ce.append(ce)
        hist_oac.append(oac)
        hist_score.append(score)
        hist_sigma.append(sigma)

        if score > best_score:
            best_score = score
            best_weights = new_weights
            best_classif = classif
            best_ce = ce
            best_oac = oac
            n_plateau = 0

```

```

        print(f"Iter {t:03d} | CE={ce:.4f} | OAC={oac:.4f} "
              f"| score={score:.4f} | sigma={sigma:.4f} ✓")
    else:
        n_plateau += 1

    # Réduction de sigma tous les 20 pas sans amélioration
    if n_plateau > 0 and n_plateau % 20 == 0:
        sigma = max(sigma * 0.5, sigma_min)

    # Redémarrage si sigma trop petit (exploration épuisée)
    if sigma <= sigma_min:
        sigma = sigma_init * 0.3
        n_plateau = 0

print(f"\n=== Résultat final ===")
print(f"OAC = {best_oac:.4f} | CE = {best_ce:.4f} | " f"score = {best_score:.4f}")

return best_classif, best_weights, hist_ce, hist_oac, hist_score, hist_sigma

```

Concernant l'algorithme de recuit simulé, je suis actuellement en train de travailler sur son implémentation. À ce stade, le code ne fonctionne pas encore et nécessite plusieurs ajustements. Je vais poursuivre le développement et les phases de débogage au cours de cette semaine. Une fois l'implémentation stabilisée, je lancerai une série de simulations afin d'analyser les résultats obtenus et d'évaluer l'apport de cette approche pour l'optimisation des poids spectraux.